



沈阳建筑大学

上机实验报告

课程名称:

操作系统

实验题目:

进程调度模拟实验

专业班级:

计算机 2101 班

学 号:

2104230414

日 期:

2024.04.03

【实验目的】

1. 理解进程的概念，熟悉进程的组成；
2. 用高级语言编写和调试一个进程调度程序，以加深对进程调度算法的理解。

【实验准备】

1. 几种进程调度算法
 - 短进程优先调度算法
 - 高优先权优先调度算法
 - 先来先服务调度算法
 - 基于时间片的轮转调度算法
2. 进程的组成
 - 进程控制块（PCB）
 - 程序段
 - 数据段
3. 进程的基本状态
 - 就绪 W（Wait）
 - 执行 R（Run）
 - 阻塞 B（Block）

【实验内容】

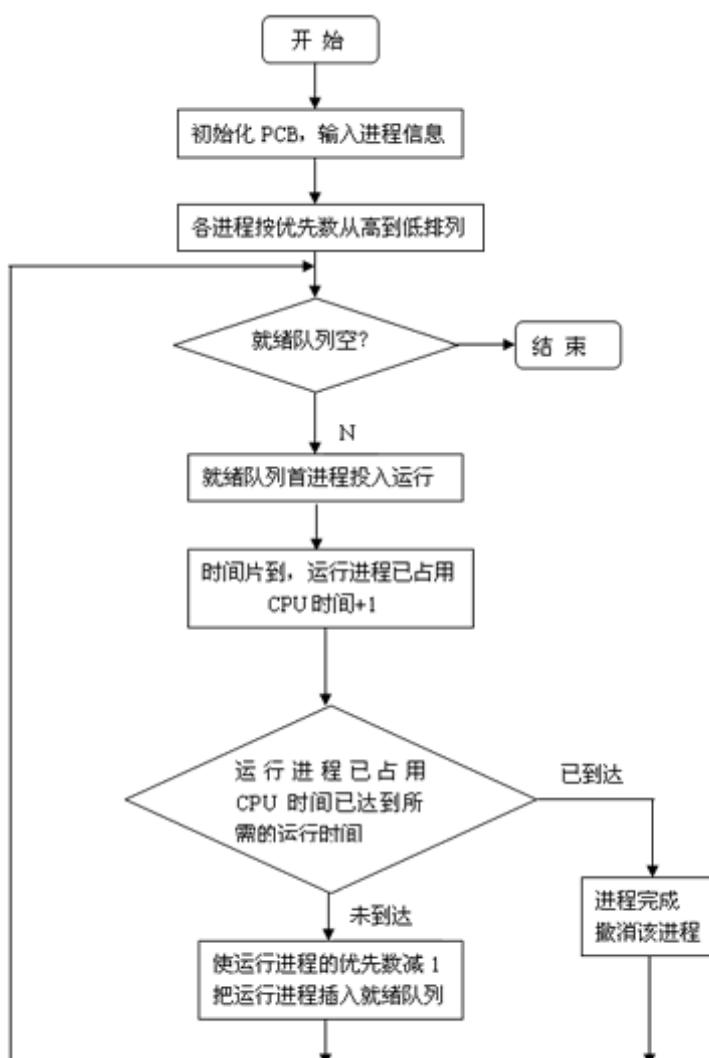
1. 例题

设计一个有 N 个进程共行的进程调度程序。

进程调度算法：采用**最高优先数优先**的调度算法（即把处理机分配给优先数最高的进程）和**先来先服务算法**。**每个进程有一个进程控制块（PCB）表示**。进程控制块可以包含如下信息：进程名、优先数、到达时间、需要运行时间、已用 CPU 时间、进程状态等等。进程的优先数及需要的运行时间可以事先人为地指定（也可以由随机数产生）。进程的到达时间为进程输入的时间。进程的运行时间以时间片为单位进行计算。每个进程的状态可以是就绪 W（Wait）、运行 R（Run）、或完成 F（Finish）三种状态之一。就绪进程获得 CPU 后都只能运行一个时间片。用已占用 CPU 时间加 1 来表示。如果运行一个时间片后，

进程的已占用 CPU 时间已达到所需要的运行时间，则撤消该进程，如果运行一个时间片后进程的已占用 CPU 时间还未达所需要的运行时间，也就是进程还需要继续运行，此时应将进程的优先数减 1（即降低一级），然后把它插入就绪队列等待 CPU。每进行一次调度程序都打印一次运行进程、就绪队列、以及各个进程的 PCB，以便进行检查。重复以上过程，直到所要进程都完成为止。

2. 调度算法的流程图



【源程序代码】

● 最高优先调度算法:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#define getpch(type) (type*)malloc(sizeof(type))
// #define NULL 0

struct pcb { /* 定义进程控制块 PCB */
    char name[10]; // 进程名
    char state; // 进程状态
    int super; // 优先级
    int ntime; // 进程需要运行的时间
    int rtime; // 已用 CPU 时间
    struct pcb* link; // 优先级队列 链表
}*ready = NULL, *p; // ready 就绪队列

typedef struct pcb PCB;

void sort() /* 建立对进程进行优先级排列函数*/
{
    PCB *first, *second;
    int insert = 0;
    if ((ready == NULL) || ((p->super) > (ready->super))) /*优先级最大者,插入队首*/
    {
        p->link = ready;
        ready = p;
    }
    else /* 进程比较优先级,插入适当的位置中*/
    {
        first = ready;
        second = first->link;
        while (second != NULL)
        {
            if ((p->super) > (second->super)) /*若插入进程比当前进程优先数大,*/
            { /*插入到当前进程前面*/
```

```

        p->link = second;
        first->link = p;
        second = NULL;
        insert = 1;
    }
    else /* 插入进程优先数最低,则插入到队尾*/
    {
        first = first->link;
        second = second->link;
    }
}
if (insert == 0) first->link = p;
}
}

void input() /* 建立进程控制块函数*/
{
    int i, num;
    system("cls");
    //clrscr(); /*清屏*/
    printf("\n 请输入进程数量: ");
    scanf("%d", &num);
    for (i = 0; i < num; i++)
    {
        printf("\n 进程号 No.%d:\n", i);
        p = getpch(PCB);
        printf("\n 输入进程名:");
        scanf("%s", p->name);
        printf("\n 输入进程优先数:");
        scanf("%d", &p->super);
        printf("\n 输入进程运行时间:");
        scanf("%d", &p->ntime);
        printf("\n");
        p->rtime = 0;
        p->state = 'W';
        p->link = NULL;
        sort(); /* 调用 sort 函数*/
    }
}
}

```

```

int space()
{
    int l = 0; PCB* pr = ready;
    while (pr != NULL)
    {
        l++;
        pr = pr->link;
    }
    return(l);
}

void disp(PCB * pr) /*建立进程显示函数,用于显示当前进程*/
{
    printf("\n qname \t state \t super \t ndtime   runtime \n");
    printf("|%s\t", pr->name);
    printf("|%c\t", pr->state);
    printf("|%d\t", pr->super);
    printf("|%d\t", pr->ntime);
    printf("|%d\t", pr->rtime);
    printf("\n");
}

void check() /* 建立进程查看函数 */
{
    PCB* pr;
    printf("\n ***** 当前正在运行的进程是:%s", p->name); /*显示当前运行进程
*/
    disp(p);
    pr = ready;
    printf("\n *****当前就绪队列状态为:\n"); /*显示就绪队列状态*/
    while (pr != NULL)
    {
        disp(pr);
        pr = pr->link;
    }
}

void destroy() /*建立进程撤消函数(进程运行结束,撤消进程)*/

```

```

{
    printf("\n 进程 [%s] 已完成.\n", p->name);
    free(p);
}

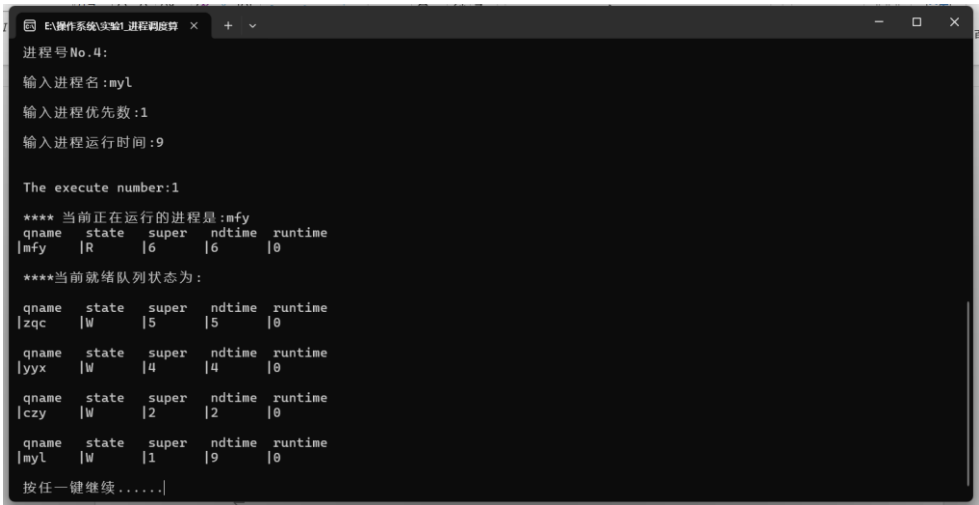
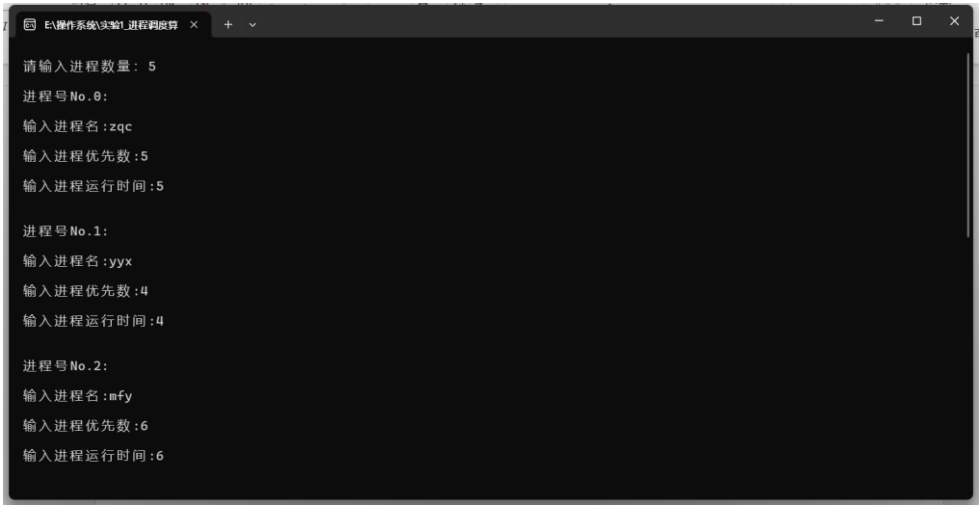
void running() /* 建立进程就绪函数(进程运行时间到,置就绪状态*/
{
    (p->rtime)++;
    if (p->rtime == p->ntime)
        destroy(); /* 调用 destroy 函数*/
    else
    {
        (p->super)--;
        p->state = 'W';
        sort(); /*调用 sort 函数*/
    }
}

int main() /*主函数*/
{
    int len, h = 0;
    char ch;
    input();
    len = space();
    while ((len != 0) && (ready != NULL))
    {
        ch = getchar();
        h++;
        printf("\n The execute number:%d \n", h);
        p = ready;
        ready = p->link;
        p->link = NULL;
        p->state = 'R';
        check();
        running();
        printf("\n 按任一键继续.....");
        ch = getchar();
    }
    printf("\n\n 进程已经完成.\n");
    ch = getchar();
}

```

```
return 0;
}
```

运行结果截图：




```
E:\操作系统实验\进程调度算  x  +  v  -  □  x
**** 当前正在运行的进程是:zqc
qname  state  super  ndtime  runtime
|zqc   |R     |2     |5     |3

****当前就绪队列状态为:
qname  state  super  ndtime  runtime
|mfy    |W     |2     |6     |4
qname  state  super  ndtime  runtime
|myl    |W     |1     |9     |0
qname  state  super  ndtime  runtime
|czy    |W     |1     |2     |1
qname  state  super  ndtime  runtime
|yyx    |W     |1     |4     |3

按任一键继续.....

The execute number:13

**** 当前正在运行的进程是:mfy
qname  state  super  ndtime  runtime
|mfy    |R     |2     |6     |4

****当前就绪队列状态为:
qname  state  super  ndtime  runtime
```

● 轮转调度算法

//RR 时间片轮转与 HPF 优先级调度算法

```
#include<iostream>
```

```
#include<cstdio>
```

```
#include<cstdlib>
```

```
#include<cmath>
```

```
#include<cstring>
```

```
#include<algorithm>
```

```
#include<queue>
```

```
using namespace std;
```

```
typedef struct{
```

```
    char pname;//进程名
```

```
    int arrive_t;//到达时间
```

```
    int service_t;//服务时间
```

```
    int finish_t;//完成时间
```

```
    int turnaround_t;//周转时间
```

```
    double w_turnaround_t;//带权周转时间
```

```
    char state;//进程状态: w: 等待, m: 就绪, r: 运行
```

```
    int priority;//优先级
```

```
    int run_t;//运行时间
```

```
    int remaining_t;//剩余时间
```

```
}PCB;
```

```
PCB p[20];
```

```
int n=10;
```

```
char op;
```

void init()//初始化进程信息： 进程名、到达时间、服务时间、状态

```
{  
    p[0].pname = 'A';  
    p[0].arrive_t = 0;  
    p[0].service_t = 3;  
    p[0].priority=2;  
  
    p[1].pname = 'B';  
    p[1].arrive_t = 2;  
    p[1].service_t = 5;  
    p[1].priority=1;  
  
    p[2].pname = 'C';  
    p[2].arrive_t = 2;  
    p[2].service_t = 2;  
    p[2].priority=4;  
  
    p[3].pname = 'D';  
    p[3].arrive_t = 4;  
    p[3].service_t = 4;  
    p[3].priority=3;  
  
    p[4].pname = 'E';  
    p[4].arrive_t = 4;  
    p[4].service_t = 1;  
    p[4].priority=10;  
  
    p[5].pname = 'F';  
    p[5].arrive_t = 4;  
    p[5].service_t = 4;  
    p[5].priority=8;  
  
    p[6].pname = 'G';  
    p[6].arrive_t = 6;  
    p[6].service_t = 2;  
    p[6].priority=9;  
  
    p[7].pname = 'H';  
    p[7].arrive_t = 6;
```

```

p[7].service_t = 6;
p[7].priority=6;

p[8].pname = 'I';
p[8].arrive_t = 6;
p[8].service_t = 5;
p[8].priority=7;

p[9].pname = 'J';
p[9].arrive_t = 6;
p[9].service_t = 2;
p[9].priority=5;

for(int i=0;i<n;i++)
{
    p[i].state='w';//初始状态都设为等待
    p[i].run_t=0;
    p[i].remaining_t=p[i].service_t;
}
}

void original_HPF_P()//输出 HPF 的初始状态
{
    printf("进程的初始状态\n");
    printf("-----\n");
    printf("进程名\t\t 到达时间\t 服务时间\t 优先级(数越大优先级越高)\n");
    int i;
    for(i=0;i<n;i++)
    {

        printf("%c\t\t   %d\t\t   %d\t\t   %d\n",p[i].pname,p[i].arrive_t,p[i].service_t,p[i].priority);
    }
}

void original_RR_P()//输出时间片轮转调度算法各进程的初始状态
{
    printf("进程的初始状态\n");
    printf("-----\n");
    printf("进程名\t\t 到达时间\t 服务时间\n");
    int i;

```

```
for(i=0;i<n;i++)
{
    printf("%c\t\t\t %d\t\t\t %d\n",p[i].pname,p[i].arrive_t,p[i].service_t);
}
}
//HPF 非抢占式优先级调度算法
//比较后面的优先级
bool cmp1(PCB a,PCB b)//先按到达时间排序，再按优先级排序
{
    if(a.arrive_t==b.arrive_t)
    {
        return a.priority>b.priority;//数越大，优先级越高
    }
    else
    {
        return a.arrive_t<b.arrive_t;
    }
}
bool cmp2(PCB a,PCB b)//按优先级排序
{
    return a.priority>b.priority;
}
void HPF_work()
{
    sort(p,p+n,cmp1);//先按到达时间排序，再按优先级排序
    p[0].finish_t=p[0].arrive_t+p[0].service_t;//第一个进程的完成时间=到达时间+服务时间
    int cnt=1;//判断完成时间内到达的进程个数,设第一个进程已经到达
    for(int i=1;i<n;i++)//看每一个进程
    {
        if(cnt<n)
        {
            for(int j=i;j<n;j++)//看后面来的进程
            {
                if(p[j].arrive_t<=p[i-1].finish_t)//比较到达时间和上一个进程的完成时间
                {
                    cnt++;//如果已经达到就标记
                }
            }
            else
```

```

        break;
    }
    if(cnt>i)//需要排序时
        sort(p+i,p+cnt,cmp2);//按优先级排序
    }
    p[i].finish_t=max(p[i-1].finish_t,p[i].arrive_t)+p[i].service_t;//完成时间

}
}

bool cmp3(PCB a,PCB b)//按到达时间排序
{
    return a.arrive_t<b.arrive_t;//按到达时间从小到大排序
}
//时间片轮转法,新来的进程应该放在就绪队列的末尾
void RR_work()
{
    int q=2;//设时间片=2
    //先按到达时间排序
    sort(p,p+n,cmp3);//把这些进程编号为 0-9
    queue<int> ready_q;//就绪队列
    ready_q.push(0);//先把第一个来的放到就绪队列
    //如果到达了，就放在就绪队列中，弹出队首元素
    int temp;
    int now_time=p[0].arrive_t;//从第一个进程到达时间开始算
    int cnt=1;//第一个进程已经处理了，接下来从 p[1]开始看。
    int i;
    printf("进程的执行顺序: \n");
    while(1)
    {
        if(!ready_q.empty())//如果不空，队列中有元素，就占用处理机，进入运行状态
        {
            temp=ready_q.front();//取队头元素
            printf("%c ",p[temp].pname);
            ready_q.pop();//弹出队首元素
            p[temp].run_t=p[temp].run_t+q;//运行时间
            now_time=now_time+q;
            //设有 10 个进程，下标是从 0-9,按顺序排好序的，所以是按顺序到达的
            while(cnt<n&& p[cnt].arrive_t<=now_time)

```

```

        {
            ready_q.push(cnt);//放入队尾
            p[cnt].state='m';//进入就绪队列
            cnt++;//等待下一个进程的到达
        }
        if(p[temp].run_t<p[temp].service_t)//如果这个进程还没有运行完成,就放到队列尾部
        {

            p[temp].remaining_t=p[temp].service_t-p[temp].run_t;//剩余时间=需要的服务时间-已经运行的时间

            ready_q.push(temp);//当前的这个再次入队, 放在队尾
            p[temp].state='m';//状态为就绪态
        }
        else
        {
            now_time=now_time-(p[temp].run_t-p[temp].service_t);
            p[temp].state='f';//已经执行完了
            p[temp].finish_t=now_time;
        }
    }
    for( i=0;i<n;i++)
    {
        if(p[i].state=='m'||p[i].state=='w')//如果还有进程是等待或者就绪态,就继续进行时间片轮转
            break;
    }
    if(i==n)//如果都执行完了
        break;
    }
    printf("\n");
}
bool cmp4(PCB a,PCB b)
{
    return a.pname<b.pname;//按进程名从小到大排序
}
void Print()
{
    int i;

```

```

if(op=='h' || op=='H')
{
    printf("进程执行顺序为: \n");
    for(i=0;i<n;i++)
    {
        printf("%c ",p[i].pname);
    }
    printf("\n");
}
sort(p,p+n,cmp4);
printf("-----\n");
printf("进程名\t\t 到达时间\t 服务时间\t 完成时间\t 周转时间\t 带权周转时间\n");
int sum_t1=0;//代表总的周转时间
double sum_t2=0;//代表总的带权周转时间
for(i=0;i<n;i++)
{
    p[i].turnaround_t=p[i].finish_t-p[i].arrive_t;//周转时间=完成时间-到达时间
    p[i].w_turnaround_t=(double)p[i].turnaround_t/p[i].service_t//带权周转时间=周转时间/服
务时间
    sum_t1=sum_t1 + p[i].turnaround_t;
    sum_t2=sum_t2 + p[i].w_turnaround_t;
    printf("%c\t\t  %d\t\t  %d\t\t  %d\t\t  %d\t\t  %0.2f\n",p[i].pname,p[i].arrive_t,
        p[i].service_t,p[i].finish_t,p[i].turnaround_t,p[i].w_turnaround_t);

}
printf("平均周转时间:%0.2f\n",(double)sum_t1/n);
printf("平均带权周转时间:%0.2f\n",(double)sum_t2/n);

}

int main()
{
    printf("请输入要使用的调度算法(<h/H:HPF> <r/R:RR>:");
    scanf("%c",&op);
    getchar();
    init();

    if(op=='h' || op=='H')
    {

```

```

        original_HPF_P();
        HPF_work();
        Print();
    }
    if(op=='r' || op=='R')
    {
        original_RR_P();
        RR_work();
        Print();
    }
    return 0;
}

```

运行结果截图：

进程的初始状态

进程名	到达时间	服务时间
A	0	3
B	2	5
C	2	2
D	4	4
E	4	1
F	4	4
G	6	2
H	6	6
I	6	5
J	6	2

进程的执行顺序：
A B C A D E F B G H I J D F B H I I

进程名	到达时间	服务时间	完成时间	周转时间	带权周转时间
A	0	3	7	7	2.33
B	2	5	27	25	5.00
C	2	2	6	4	2.00
D	4	4	24	20	5.00
E	4	1	10	6	6.00
F	4	4	26	22	5.50
G	6	2	16	10	5.00
H	6	6	33	27	4.50
I	6	5	34	28	5.60
J	6	2	22	16	8.00

平均周转时间:16.50
平均带权周转时间:4.89

【实验体会】

通过第一次的上机，我理解个多种进程调度算法，通过用代码去实践，也理解了各个进程调度算法细微的区别。包括高优先级调度算法和轮转调度算法，增加了自己对调度算法实际实践的理解，不同的调度算法没有具体的优劣之分，要看使用的场景。

【思考题】

几种进程调度算法各有什么优缺点？

1.时间片轮转调度算法（RR）

优点：确保所有进程都能在一定时间内获得 CPU 时间，提高了系统的整体响应速度，特别适合于交互式系统。

缺点：如果时间片设置不当，可能会导致系统响应时间变差或者

CPU 效率降低；对于长时间运行的进程可能造成延迟。

2.先来先服务调度算法（FCFS）

优点：实现简单，公平，每个进程按到达顺序执行。

缺点：不利于短进程，可能导致平均周转时间较长，特别是当短进程在长进程之后到达时。

3.优先级调度算法（HPF）

优点：能有效处理紧急或重要的进程，提高了系统对高优先级任务的响应能力。

缺点：可能存在低优先级进程饥饿的问题，即长时间得不到执行的机会；优先级反转现象也可能发生，即低优先级进程阻塞高优先级进程。

4.多级反馈队列调度算法

优点：综合了多种调度策略，能够根据进程的运行特性将其放入不同的队列，提高了系统的适应性和灵活性。

缺点：实现较为复杂，需要维护多个队列及相应的转换规则，可能会引入额外的开销。

5.高响应比优先调度算法（HRRN）

优点：兼顾了公平性和效率，通过计算响应比（ $R = (\text{等待时间} + \text{运行时间}) / \text{运行时间}$ ）来决定下一个执行的进程，既照顾到了短进程，也考虑了长进程的等待时间。

缺点：需要预知或估算进程的运行时间，这在实际中可能难以准确获取；算法的计算复杂度相对较高。